

# Evidence Crosses, Authority Does Not: Receiver-Owned Admission of Agent Tool Calls Within and Across Organizations

The Chio Project

<https://github.com/bb-connor/chio>

September 2026

## Abstract

An agent that calls a tool through an ordinary client holds whatever authority its host process holds and leaves whatever record that process chooses to keep, and when the call crosses into a second organization the receiver has nothing to check but the sender's word. We describe a protocol built on one rule: authority is minted and verified only inside the kernel that dispatches, against roots that kernel activated itself, and nothing in a request can add a root; only signed, content-addressed evidence crosses an organization boundary. The object that crosses is the receipt. Locally it is a kernel-signed record of one admission decision whose identity is the hash of its own content, written durably before the caller sees a result and, when the exercised grant carries a monetary cap, carrying what the call cost and who paid. Across a boundary it is co-signed: a statement whose single subject is the digest of a receipt the receiving kernel already holds, signed by exactly two keys the receiver pinned before the request arrived, binding the agreement, the request hash, and a single-use continuation. The statement is a pointer, not a bearer credential, so every deviation is a hash mismatch, a signature failure, or a lookup that returns nothing.

## 1 The rule

A buyer agent at one company has read a refund case and confirmed the customer, and it asks a vendor agent at a second company to stage the refund against the vendor's payment ledger. The vendor's software must decide, before it touches that ledger, whether to obey. Today that decision rests on an API key and a transport signature, which establish who is calling and nothing about whether this call, with these arguments, under the agreement these two companies signed, was already authorized on the vendor's side, already performed, or rests on a delegation revoked four minutes ago. Afterward each company holds a log it wrote about itself, and no third party can tell whether the two describe the same event. Having the sender prove more makes the receiver's security a function of the sender's correctness, and a shared ledger or broker makes that substrate a participant in every call.

The rule here avoids both. **Authority is minted and verified only inside the kernel that dispatches, against roots that kernel activated itself, and nothing carried in a request can add a root. Only signed, content-addressed evidence crosses an organization boundary.** A signature proves who signed some bytes. It cannot decide whether the receiver should act on them, because that decision depends on facts that exist only at the receiver: which keys it pinned, which agreement it activated and when, whether this continuation has been spent.

That is a placement argument; it composes with the rest of the stack rather than replacing it [16]. Alignment training, transcript monitors, and the sender's own policy engine change how often the kernel says no; none can know that the callee's budget is exhausted or that this argument hash was already dispatched. The receiver decides.

The receipt is the one object this paper asks a reader to hold. Locally a **receipt** is a kernel-signed record of one admission decision whose identity is the hash of its own content; across a boundary the same object is **co-signed**, a statement whose single subject is the digest of a receipt the receiving kernel already holds, signed by exactly two keys the receiver pinned before the request arrived. A cross-organization call is that admission operation with the peer's receipt and co-signature supplied as evidence, never as authority, which makes the statement a pointer rather than a bearer credential: holding it grants nothing, because everything it names must already be in the receiver's stores, and no deviation is one the sender can repair by being more persuasive. Agent protocols define how a tool is called and how agents talk [2, 1]; neither defines the record of what was admitted. We call the mediating component the kernel; the implementation measured here, Chio, is userspace and not an operating-system kernel.

## 2 Threat model and trusted base

The kernel sits between an agent and the tool servers whose transport it owns: child processes it spawned and speaks to over their standard streams, HTTP tools it fronts, and sidecars it proxies. It does not confine the agent: the model, the orchestrator, the prompt, and any library the agent imports are untrusted, and an agent that opens its own socket reaches whatever that socket reaches.

The adversary is given everything outside the receiving kernel. It controls every request field this paper names; it controls the peer kernel and can make it sign anything with the peer’s key; it controls the network and may drop, delay, reorder, duplicate, and replay; and it may present any receipt, statement, lease, or governance record it has ever observed, in any combination, to any kernel. It does not hold the receiving kernel’s signing key and cannot write to its stores. The question is what such an adversary can cause a receiving kernel to dispatch.

The trusted base is the receiving kernel’s process, its signing key, and its stores; everything outside it is verified or assumed. The assumptions appear once, in Table 1.

Table 1: Assumptions outside the receiving kernel, by the identifiers of the implementation’s assumption registry. The registry has 16 entries; the ten this protocol rests on are listed.

| Identifier                             | What is assumed                                                                                                                                                                                     |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ASSUME-ED25519                         | Ed25519 meets existential unforgeability under chosen-message attack for keys the kernel trusts.                                                                                                    |
| ASSUME-SHA256                          | SHA-256 is collision resistant over receipt and statement preimages.                                                                                                                                |
| ASSUME-CANONICAL-JSON                  | The production canonical encoder agrees byte for byte with the mechanized one on strings, bounded integers, finite arrays, and objects ordered by UTF-16 code unit. Floats are outside that domain. |
| ASSUME-SQLITE-ATOMICITY                | A single-row insert commits atomically, so a primary-key conflict is a reliable witness that a row existed. Cross-row recovery and ordering are not assumed.                                        |
| ASSUME-OS-CLOCK                        | The injected clock reports Unix time within the operator’s declared tolerance.                                                                                                                      |
| ASSUME-NETWORK-TRANSPORT, ASSUME-TLS   | Delivery is not assumed reliable; arriving messages are assumed not rewritten below the signature checks, and remote surfaces authenticate their endpoints.                                         |
| ASSUME-SUBPROCESS-ISOLATION            | Effects after an allow, and isolation between tool processes, rest on operating-system boundaries.                                                                                                  |
| ASSUME-FEDERATED-ORIGIN-CLASSIFICATION | The receiving edge marks cross-organization origin from a transport-authenticated peer identity; a request misclassified as local never reaches these checks.                                       |
| ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND | Revocation liveness only: recurring delivery under weak fairness, bounded skew, operator-bounded partition healing. Safety does not depend on it.                                                   |

One assumption is deliberately absent: we do not assume the two signing keys on a co-signed statement are held by two different organizations, because one operator holding both satisfies every check in this paper. Key independence is a deployment property and the protocol carries no evidence of it.

## 3 The receipt

A receipt is a JSON object with a fixed field set, encoded canonically under RFC 8785 [15]. Let  $b$  be the receipt without its `signature`, without its `id`, and without the two optional fields that describe how it was signed, and write **body** for  $b$  together with its `id`, the canonical receipt body the verifier of Section 5 resolves. Two identities define the object, both checkable with a canonicalizer, SHA-256, and Ed25519 [12]:

$$id = \text{SHA-256}(\text{JCS}(b)), \quad \sigma = \text{Ed25519}_k(\text{JCS}(\{\text{body}: b, \text{id}: id\}))$$

where  $k$  is the kernel key published in the body and the literal JSON key `body` carries  $b$ . A receipt renamed is therefore a receipt unsigned. Three properties follow.

**Recompute and refuse.** On every path that holds the evaluated content the signer is handed the content, not a digest: it recomputes the digest inside the trust boundary and refuses to sign if the body’s claimed digest differs.

**Decisions, not successes.** A denial is a receipt of the same shape, built, signed, and persisted on the same path as an allow. Three fields fix what kind of statement a receipt is, so a weaker one cannot pose as a stronger one: `receipt_kind` separates a mediated decision from a passive trace, `boundary_class` says whether the kernel could have prevented the effect or only observed it, and `trust_level` records the evidence class. A body carrying a decision on a kind that cannot make one will not be signed, and a message rejected before it had a decision to record returns its code and no receipt.

**Permission, proof, and price in one object.** When the exercised grant carries monetary caps the hold taken before dispatch is reconciled into the receipt’s financial block: `cost_charged`, `currency`, `budget_remaining`, `budget_total`, the delegation depth, and the root budget holder. The receipt an auditor reads to learn the call was allowed is the object a biller reads to learn what it cost. Settlement between organizations is out of scope, and that is the whole of our claim about commerce.

The vendor kernel’s receipt for the refund step, abridged at the metadata:

```
{ "id": "d342a59d...6fba2e3f", "timestamp": 1766000001,
  "capability_id": "lease-vendor-c-refund", "tool_server": "vendor-c.payments",
  "tool_name": "stage_refund",
  "action": { "parameters": { "caseRef": "refund-250", "tool": "stage_refund",
    "workflowId": "wf-chio-refund-001" },
    "parameter_hash": "47e6e096...d5a6464b" },
  "decision": { "verdict": "allow", "trust_level": "mediated",
    "receipt_kind": "mediated_decision", "boundary_class": "prevent",
    "tool_origin": "caller_executed", "redaction_mode": "none",
    "content_hash": "c9abf68a...d9803987",
    "policy_hash": "82cfeae63...fb2ca3fd",
    "metadata": { "attribution": { "delegation_depth": 0, "grant_index": 0, ... },
      "chio_runtime": { "accepted": true, "destructive": true,
        "lease_id": "lease-vendor-c-refund", "step_index": 2,
        "reserved_treaty_continuation_id": "continuation:runtime-loopback:2", ... },
      "kernel_key": "31debe55...cfa498bc",
      "signature": "a9e4762e...1ba28505" }
```

The identity rule can be checked on these bytes. Canonicalized without `signature` and without `id` the object hashes to `d342a59d...6fba2e3f`, its `id`; without `signature` alone, to `31118bb6...4781b3f9`, the subject of the co-signed statement in Section 5; whole, to `63f1f35a...8c93e3e8`, what the binding reference calls the remote receipt digest. Wrap the first preimage with the `id` in the two-key object and verify under the embedded kernel key: it verifies.

This receipt carries no financial block; the cost sits in the signed workflow receipt’s record for step 2, 250 of 550 USD minor units, and a grant with its own cap puts the block in the tool receipt instead.

## 4 Admission

Admission is the one operation. Every tool call, local or foreign, goes through it, and every path out of it that reached a decision produces a signed receipt.

1. Verify the presented capability against the kernel’s own trusted issuer set: signature, time window, subject binding, and the revocation state of the leaf and every ancestor in the presented chain; resolve the grants matching this tool and these arguments. Nothing in the request extends that set.
2. Open a durable admission operation, a fenced write-ahead record whose state machine has 18 states, from prepared to a terminal classification. The fence is a store lease with an epoch, so a second coordinator cannot advance the same operation.
3. Run the guard pipeline. Deny dominates, and so does error: a guard that fails to produce a verdict is a denial, not a pass.
4. Run the admission hook, where Section 5 runs for a request classified as coming from another organization; such a request carrying no agreement context is denied here.
5. Take the budget hold, charging the reservation before dispatch.
6. Revalidate the mutable inputs immediately before dispatch, mark the operation dispatch-committed, and only then hand the call to the tool server over a transport the kernel owns.
7. Run output guards, build the receipt, recheck its fields against the admitted inputs, sign it, persist it, and only then return.

Two orderings are the whole of its value. **Durable before dispatch:** no dispatch occurs without a committed pre-dispatch record. Before that commit recovery compensates, releasing the hold; after it, with no outcome known, recovery cannot invent one, so the operation terminates as outcome-unknown, the honest answer and the one a reconciling system can act on. An output that does not match the committed digest terminates as denied-after-delivery, with a signed deny persisted and nothing captured against the hold. Every crash is classified rather than lost, in the signed record.

**Receipt before response:** the receipt is persisted locally before the caller sees the result, and a kernel that cannot persist a receipt denies. The log is a write-ahead record of what was admitted, not a trace of what was observed. Denials release what they did not use, scoped to the admission that took the reservation, and an unconfirmed release is recorded as ambiguous rather than released.

## 5 Cross-organization admission

A call from another organization is the same admission operation. The peer kernel's receipt and co-signature enter at step 4 as evidence and add no authority: the receiver's issuer set, pinned peers, agreements, and clocks are what they were before the request arrived. Figure 1 is the whole of what a statement can name.

### 5.1 Receiver-owned resolution

Two organizations pin each other out of band, each recording the other's kernel identity, the passport key that kernel co-signs with, and a rotation deadline. The agreement is an activated record naming exactly two kernels, the action classes in scope, a validity window, and a revocation epoch. Each participant publishes a signed manifest, the action classes it accepts and at what intensity on the ascending ladder **observation**, **guarded**, **receipt\_backed**, **partition\_contingency**, **maintenance**, and each receiver activates the agreement and both manifests locally and intersects the two per action class itself: the higher-intensity mode wins, the stricter co-signing requirement wins, a destructive class below **receipt\_backed** rejects, and mismatched consistency models reject. Consistency is a separate axis, not a mode. Continuations resolve by identifier against the receiver's own copy. A pin the receiver has not activated is a denial, not a prompt.

The request may carry identifiers. It may not carry trust. Its agreement context is names and digests, each a lookup key into a store the receiver owns: the agreement and the intersection with the digest the receiver should find for each, the action class, and identifiers for the continuation, the lineage bundle that chains this call to the receipts behind it, and the co-signed evidence. Eleven further key names are refused wherever they appear. Seven name a trust root, a bundle, an agreement record, a manifest, a signing key, or a peer directory, refused with `request_smuggled_trust_root`: `trustRoot`, `trustRoots`, `trustBundle`, `treatyScope`, `ladderManifest`, `signingKey`, `peerDirectory`. Four name dynamic trust input or peer discovery, refused with `request_smuggled_dynamic_trust`: `dynamicTrust`, `dynamicTrustBundle`, `runtimeTrustInput`, `peerDiscovery`. The check is on names, runs before anything is resolved, and is the operational form of the rule.

### 5.2 The co-signed statement

The statement is an in-toto Statement v1 [21] in a DSSE envelope [18] whose predicate is `snake_case` and rejects unknown fields. Its single subject names the invocation identifier, which is the receipt's `id`, and carries as its digest the SHA-256 of the canonical receipt body of Section 3. The envelope for the refund step, then its payload, abridged from the committed evidence package for workflow `wf-chio-refund-001`:

```
{ "payloadType": "application/vnd.in-toto+json",
  "payload": "eyJfdHlwZSI6Imh0dHBzOiI8vaW4tdG90by5pby9T...",
  "signatures": [
    { "keyid": "fdf72a08f18f7399e8c52bce448441501f759a595a86980e5d9a422a01e5d55",
      "sig": "3QZ09JP4YLFmvgzwy5PwqYTWfCcQkUo/DR07YyFB8pD+bA6iBGwyRTtI+lzzI7A..." },
    { "keyid": "e37abcd79efcd08db94518a903952f9e9f8af7e8b4684b63591631cef8e8871c8",
      "sig": "R0Ii8tcBzYcEPAPRQqNmsbCUowFH0hzpjr5Mm98mFu33+XcT0fDxdWf48btYzmd..." } ] }
```

```
{ "_type": "https://in-toto.io/Statement/v1",
  "predicateType": "chio.bilateral-cosign-invocation.v1",
  "subject": [ { "name": "chio-receipt:d342a59d...6fba2e3f",
                  "digest": { "sha256": "31118bb6...4781b3f9" } } ],
  "predicate": {
    "invocation_id": "d342a59d...6fba2e3f", "tool_name": "stage_refund",
    "tool_server_a": { "kernel_id": "did:chio:buyer-kernel", "alg": "ed25519",
      "passport_key_fingerprint": "fdf72a08...a01e5d55" },
    "tool_server_b": { "kernel_id": "did:chio:vendor-c", "alg": "ed25519",
      "passport_key_fingerprint": "e37abcd7...8e8871c8" },
    "co_sign": "bilateral_required", "consistency_model": "totally-ordered",
    "consistency_anchor": "chio:consistency:wf-chio-refund-001:2",
    "cross_org_visibility": "federated", "timestamp_unix_ms": 1766000001000,
    "tool_args_hash": { "alg": "sha256", "value": "47e6e096...d5a6464b" },
    "capability_lease_ref": { "lease_id": "lease-vendor-c-refund",
      "issuer": "did:chio:buyer-kernel", "expires_at_unix_ms": 1766000030000,
      "scope_digest": { "alg": "sha256", "value": "262768fe...b9f835aa" } },
    "governance_receipt_ref": { "receipt_id": "gov-refund-stage-authorization",
      "kernel_id": "did:chio:buyer-governance",
      "digest": { "alg": "sha256", "value": "21143a93...f161e3c0" } },
    "policy_evaluation_summary": { "joint_disposition": "allow",
      "server_a_verdict": { "verdict": "allow", "policy_id": "buyer-policy:stage_refund", ... },
      "server_b_verdict": { "verdict": "allow", ... } },
    "treaty_binding_ref": {
      "treaty_id": "treaty:runtime-loopback:2",
      "action_class_id": "workflow.cross_kernel.stage_refund",
      "consistency_model": "totally-ordered",
      "treaty_scope_sha256": "0ff230e9...d1f2b21c",
      "ladder_intersection_sha256": "93dcc7cf...af79d3d1",
      "admission_report_sha256": "91d74509...1f646522",
      "continuation_sha256": "8ef2f348...366b93f4",
      "lineage_bundle_sha256": "28d816a3...d3b504ba",
      "request_sha256": "47e6e096...d5a6464b",
      "outcome_sha256": "c9abf68a...d9803987",
      "local_receipt_sha256": "9cbe4438...ad017121",
      "remote_receipt_sha256": "63f1f35a...8c93e3e8",
      "lease_refs": [ "lease-vendor-c-refund" ],
    }
  }
```

Table 2: The conforming verifier, 26 steps. The receiver’s own state decides everything outside steps 8 to 14, which are the envelope layer, called with the two keys the receiver pinned.

| #     | Check                                                                                                                                                                                                                                                                              | Code on failure                                                                                                                                                                                                                                      |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1–5   | Media type, at least one signature, payload decodes and parses; predicate type <b>chio.bilateral-cosign-invocation.v1</b> ; Statement v1; exactly one subject                                                                                                                      | <b>dsse.malformed</b> ,<br><b>statement.malformed</b> ,<br><b>predicate.type_unrecognised</b> ,<br><b>statement.schema_invalid</b>                                                                                                                   |
| 6–7   | Both kernel identifiers are in the pinned peer set and each pin’s fingerprint equals the declared one                                                                                                                                                                              | <b>peer.unpinned_or_keyid_mismatch</b>                                                                                                                                                                                                               |
| 8–13  | Exactly two signatures; re-canonicalizing reproduces the payload bytes; predicate schema and strict rules; both verdicts present, equal, in {allow,deny}; subject name is <b>chio-receipt</b> ; plus the invocation identifier; the two pinned keys and their identifiers distinct | <b>statement.malformed</b> ,<br><b>predicate.schema_invalid</b> ,<br><b>policy.verdict_disagreement</b> ,<br><b>subject.digest_mismatch</b> ,<br><b>signer.independence_required</b><br>at the envelope layer, surfacing as<br><b>dsse.malformed</b> |
| 14    | <i>Only now</i> : compute the pre-authentication encoding, select each signature by a pinned key’s identifier, and verify both                                                                                                                                                     | <b>signature.server_a_invalid</b> ,<br><b>signature.server_b_invalid</b>                                                                                                                                                                             |
| 15–16 | Each pinned peer’s manifest reference is present and fresh, and the peer is active at the pinned epoch height                                                                                                                                                                      | <b>ladder.manifest_missing</b> ,<br><b>ladder.manifest_stale</b> ,<br><b>peer.revoked_at_epoch</b>                                                                                                                                                   |
| 17–19 | Resolve the receipt by invocation identifier in the receiver’s own store; its signature verifies, its tool name and argument hash agree, its kernel key is the pinned second-party key, and subject name and digest match the resolved body                                        | <b>subject.digest_mismatch</b> ,<br><b>peer.unpinned_or_keyid_mismatch</b>                                                                                                                                                                           |
| 20–22 | Both verdicts resolve and agree; the lease resolves with matching issuer, expiry and scope digest and is unexpired; the tool name is in the action-class table; a receipt-backed class resolves its governance record                                                              | <b>policy.verdict_disagreement</b> ,<br><b>capability.lease_expired_or_unknown</b> ,<br><b>governance.unknown_action_class</b> ,<br><b>governance.receipt_required_missing</b>                                                                       |
| 23–26 | Binding cross-checks (Table 3), consistency model and anchor, co-signing mode; return the verified material                                                                                                                                                                        | <b>predicate.schema_invalid</b>                                                                                                                                                                                                                      |

```
"governance_refs": [ "gov-refund-stage-authorization" ],
"signer_kernel_ids": [ "did:chio:buyer-kernel", "did:chio:vendor-c" ] } }
```

Both signatures cover the DSSE pre-authentication encoding of the payload, which here begins DSSEv1 28 `application/vnd.in-toto+json 2972 {"_type":"https:` and runs to 3016 bytes; over those bytes both verify, one under each kernel’s key, and each `keyid` is the SHA-256 of its raw public key. Every identity above was recomputed from the committed bytes: `outcome_sha256` is the receipt’s `content_hash`, and `request_sha256` is both `tool_args_hash.value` and the receipt’s `action.parameter_hash`.

### 5.3 The verifier, in the order it runs

Table 2 is the shipped order. Two components decide, in two disjoint code families. The conforming verifier returns the specification’s dotted codes, a closed set of 24 strings of which it emits 16 and the envelope layer the rest. The kernel’s pre-dispatch hook, which resolves the same evidence inline during admission, answers from the kernel’s runtime failure-code set of 505 strings, 24 of them agreement-scoped, and maps every envelope-layer failure to `chio_treaty_unverified_required_evidence`. Table 3 and the denial experiment of Section 8 report the hook, because the hook decides a live call.

Signatures are verified last and the pin lookup first, the opposite of the usual advice: the verifier cannot verify until it knows which keys to verify under, and it takes those from its own pin set, never from the envelope, so a signature whose key identifier matches no pinned key is a missing signature. No gate before step 14 consults a secret, so the ordering costs nothing in soundness. The typed error exposes a code and keeps the rendered detail for the local log, since that message names presented and expected digests and would make the verifier an oracle.

### 5.4 Consume, bind, co-sign

Replay is not defeated by a signature, since a replayed statement is validly signed. It is defeated by a single-use continuation: a token minted by the origin binding the parent receipt, the parent session anchor, the capability, the action class, the target tool, a nonce, the two kernel identities, and a validity window, whose digest is one of the 15 bound fields. The receiver’s state machine over it is two operations on one table keyed by the continuation identifier: *consume* inserts, ignoring a conflict, and reads the affected row count; zero rows means the continuation was already spent and the call is denied with `chio_treaty_continuation_replay`. *Release* deletes the row for that identifier and that admission, and runs only when a later pre-dispatch step

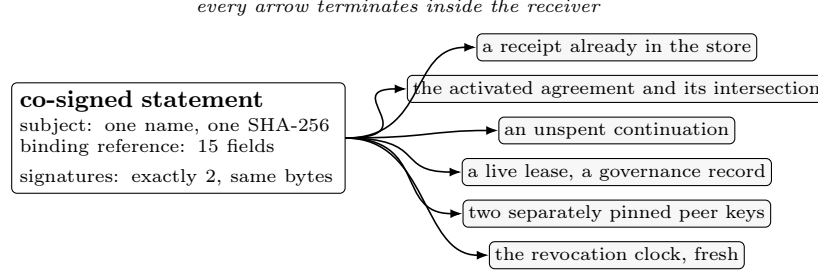


Figure 1: The mechanism. The statement carries no authority of its own: its subject, the binding fields the receiver compares, and each of its two keys resolve only against records the receiving kernel placed in its own stores before the request arrived.

denies, so a rejected call does not burn a continuation the origin will retry. Once dispatch commits the row stays: at-most-once, on purpose.

The receiver then builds its own admission report and binds the accepted material into the receipt it is about to sign. One field is treated differently from the other fourteen: the receiver checks only that `admission_report_sha256` is well-formed hex, then recomputes the digest of its *own* report and overwrites the field before signing, because a peer-supplied hash must never sit inside a locally signed receipt as though the receiver had computed it. Verified material is installed for that one request as a validated value, never as configuration. The local receipt is persisted; only then does the kernel ask the peer to co-sign a statement whose subject is the new receipt’s digest, so a refusal or a transport failure is recorded against a receipt that already exists rather than erasing it.

## 6 Security

Four properties carry the protocol, each stated over the adversary of Section 2 and reduced to an assumption in Table 1.

**Admission binding.** *If a receiving kernel dispatches under cross-organization admission, then there is a statement  $S$  such that both keys the receiver pinned for this agreement signed the identical pre-authentication bytes of  $S$ ; the subject of  $S$  is the digest of a receipt the receiver resolved from its own store; every binding field of  $S$  that the receiver compares equals the value the receiver computed for this request from its own state; and the continuation named by  $S$  was unconsumed when admission began.* Suppose not. Each comparison is byte equality between a digest the receiver computed and one the statement carries, so a difference that passed means two distinct objects share a SHA-256 image, contradicting ASSUME-SHA256, or share a canonical encoding, contradicting injectivity of the encoder under ASSUME-CANONICAL-JSON. A difference in a signature means one verified under a pinned key over bytes that key never signed, contradicting ASSUME-ED25519; one in the continuation means a single-row insert reported a create for a key that already had a row, contradicting ASSUME-SQLITE-ATOMICITY. What remains is the completeness of the comparison set, and that is Table 3. The property holds as stated for an action class that resolves the receipt lineage; without one the comparison that binds the lineage bundle does not run, and the table says which.

**Receiver locality.** *The trust roots, pins, agreements, action-class tables, and registries used to decide a request are a function of the receiver’s state before the request arrived, and are independent of the request’s content.* This one is syntactic rather than cryptographic: the eleven key names of Section 5 are rejected before any resolution runs, and what remains are identifiers, each the argument of a total lookup over receiver-owned state.

**Single use.** *For a continuation identifier  $c$ , at most one admission ever reaches dispatch with  $c$  bound to it.* Under ASSUME-SQLITE-ATOMICITY exactly one of any set of concurrent inserts on a primary key creates the row; every other reports zero created and denies. Release deletes only when keyed by both  $c$  and the admission that took it, so no admission frees another’s row and no post-dispatch path frees one at all.

**Audience binding.** *A statement accepted by receiver  $R$  is not accepted by a different receiver  $R'$  unless  $R'$  separately pinned the same two keys, activated the same agreement, and holds the same receipt.* The statement names the two kernels, but no receiver takes its audience from that name: the identifier is only an

index into the pin set, so rewriting it fails the pin lookup before any signature is checked.

**Substitution over the binding reference.** Table 3 is an executed corpus over the 15 binding fields, evidence PS-T12: a case builds a complete admissible bundle, substitutes one field with a well-formed different value, re-signs the statement with both participant keys, asserts both signatures verify over the substituted bytes, and drives it through the pre-dispatch hook, asserting the outcome the field’s row records: for a compared field the failure code and that no tool was dispatched, and for the one field the receiver never compares, admission. Of the 20 cases one substitutes nothing and admits, so every denial is attributable to the field that changed. There `binding_mismatch` and `unverified_evidence` abbreviate `chio_treaty_dsse_binding_mismatch` and `chio_treaty_unverified_required_evidence`.

Table 3: Substituting one binding field at a time, with both signatures valid over the substituted statement, as decided by the pre-dispatch hook. Ten fields are compared on every cross-organization admission, the signer list twice, as a set and in order. Four more are compared only when the admission resolves the artifact they bind, which the action class’s required-evidence set decides; the refund call resolves both the invocation record and the lineage bundle. The fifteenth is never compared, and the case that substitutes it is admitted.

| Field                                     | Receiver-side value it must equal                                                                                                                                           | First code                                            | Compared when    |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|------------------|
| <code>treaty_id</code>                    | the agreement the receiver resolved for this request                                                                                                                        | <code>binding_mismatch</code>                         | always           |
| <code>treaty_scope_sha256</code>          | the digest the receiver computes over its own agreement record                                                                                                              | <code>binding_mismatch</code>                         | always           |
| <code>ladder_intersection_sha256</code>   | the intersection digest the receiver resolved                                                                                                                               | <code>binding_mismatch</code>                         | always           |
| <code>admission_report_sha256</code>      | <i>nothing</i> : 64 hex characters, then overwritten by the receiver’s own report digest                                                                                    | none: the substituted call is admitted and dispatches | never            |
| <code>continuation_sha256</code>          | the resolved continuation’s digest                                                                                                                                          | <code>binding_mismatch</code>                         | always           |
| <code>action_class_id</code>              | the action class the receiver resolved for this request                                                                                                                     | <code>binding_mismatch</code>                         | always           |
| <code>consistency_model</code>            | the model the resolved intersection fixed                                                                                                                                   | <code>binding_mismatch</code>                         | always           |
| <code>request_sha256</code>               | this dispatch’s argument hash, and the predicate’s own <code>tool_args_hash</code>                                                                                          | <code>binding_mismatch</code>                         | always           |
| <code>lease_refs</code>                   | the lease in the receiver’s own admission bundle                                                                                                                            | <code>binding_mismatch</code>                         | always           |
| <code>governance_refs</code>              | the governance record in that bundle                                                                                                                                        | <code>binding_mismatch</code>                         | always           |
| <code>signer_kernel_ids</code> , as a set | exactly two, equal as a set to the agreement’s participants                                                                                                                 | <code>binding_mismatch</code>                         | always           |
| <code>signer_kernel_ids</code> , in order | the invocation record’s order; a reversal resolves the two pinned keys transposed, and strict envelope verification refuses that pairing before the ordered comparison runs | <code>unverified_evidence</code>                      | always           |
| <code>lineage_bundle_sha256</code>        | the lineage bundle the receiver resolved                                                                                                                                    | <code>binding_mismatch</code>                         | lineage required |
| <code>outcome_sha256</code>               | the outcome digest in the receiver’s invocation record                                                                                                                      | <code>binding_mismatch</code>                         | record required  |
| <code>local_receipt_sha256</code>         | the lineage bundle’s root receipt digest, and the invocation record’s                                                                                                       | <code>binding_mismatch</code>                         | either required  |
| <code>remote_receipt_sha256</code>        | the lineage bundle’s leaf receipt digest, and the invocation record’s                                                                                                       | <code>binding_mismatch</code>                         | either required  |

A class in the bilateral-required co-signing mode has the invocation record forced into its required-evidence set, so the last three rows run on every admission under that mode, and two named signers resolving to equal public keys are refused before any signature is checked. `admission_report_sha256` is outside the property: it is never compared, and the case that substitutes it is admitted and dispatches. Nothing rests on it, because the receiver overwrites it with its own report digest before signing, so the peer’s value reaches neither the decision nor the receipt. The interesting attack is substituting the whole statement for another validly co-signed statement from the same pair of kernels, and it is the continuation, not any digest, that stops it: the substituted statement names a continuation the receiver either does not know or has already consumed.

Two quantities follow from the table’s shape, not from measurement. The evidence a receiver resolves is fixed by the fields of one binding reference rather than by the size of the request: exactly what Figure 1 shows. The one term that grows is the lineage bundle, whose edges the receiver walks when the action class requires lineage, and that walk is the only part of admission linear in anything the request presents. The window in which a statement can be presented is the intersection of its lease, governance, and continuation validity intervals; in the refund example the lease runs 60,000 ms and the authorization 40,000 ms, so the window is at most 40 s, and after the first dispatch it is zero.

**Revocation.** Revocation is the only part of this protocol that depends on something arriving. Each origin publishes a signed root carrying a monotone epoch counter and an issue time. A kernel installs a snapshot only if its epoch strictly exceeds the installed one, by compare-and-swap, so a replayed or rewound frame cannot take effect and a concurrent install cannot be lost, and it applies its own freshness window to that snapshot’s issue time before trusting it. Safety holds unconditionally, in every partition, because both halves are local predicates that never wait for a message: no call is allowed against a revocation the kernel has observed, and a call evaluated against a stale snapshot is denied. There is no third branch.

Liveness holds only under ASSUME-GOSSIP-FAIRNESS-PARTITION-BOUND. We bound the time before a revocation is observed, not the number of evaluations, which is unbounded. Let  $F$  be the freshness window,  $T$  the origin’s control-poll period,  $L$  the latency of one admitted call, and  $k$  the number of calls made before one is decided against the newer snapshot, one at best and more when a call already in flight was decided against the older. A still-connected receiver denies no later than  $T + kL$  after publication, and a cut receiver no later than  $F + T + kL$ , because its snapshot must first age past  $F$ . With  $F = 500$  ms,  $T = 250$  ms,  $k = 1$ , and the median  $L$  of Section 8, those are 456.8 ms connected and 956.8 ms cut. An operator sizing  $F$  is choosing how long a withdrawn capability may still be honored on a failed link.

## 7 What is mechanized

The decision core is a pure function with no input or output of its own: verify a capability, resolve the matching grants, evaluate, sign. It is modeled in Lean 4 [8], and the published theorem inventory carries 183 declarations and one explicit axiom, a symbolic collision-resistance idealization registered against ASSUME-SHA256. No serializer, protocol step, or kernel behavior is axiomatized; an unproved property is an assumption-registry entry, not a hole in the development.

Three results bear on this paper, all bounded. A finite-domain refinement checker is sound and exact: it succeeds exactly when every receipt in the supplied finite domain admitted under the amended agreement was admitted under the prior one. Occurrence theorems make the predicate language fail closed: an atom that is unsupported, or undefined for the admission view, anywhere in a predicate forces denial on every view, by induction over the predicate. An accept-set decomposition shows envelope acceptance is exactly issuer-key trust, kernel-key trust, and satisfaction of the fail-closed predicate, with signature validity abstracted into trust-store membership.

**Lean does not verify the running Rust.** That is the boundary. What exists instead is four kinds of tripwire, each failing loudly when implementation and model drift apart: named Rust functions mirrored to named Lean definitions by content hash and checked on every change; 20 production functions extracted through a Rust-to-Lean pipeline with equivalence theorems that must rebuild byte-identically; 47 bounded model-checking harnesses over public entry points, among them ones asserting that an untrusted issuer is rejected before dispatch and that the signer refuses a content-digest mismatch; and the cross-language encoder corpus of Section 8. Distributed revocation has a bounded model over lossy, duplicating, reordering channels with skew and partition.

We report mutation survivors rather than percentages. Of 45 mutants introduced into the Lean development 33 were killed and 12 survived, each with an open issue; of 166 introduced into the Rust proof-facing sources 160 were killed, 5 were unviable, and 1 survived. A survivor is a place where some theorem is weaker than it looks.

## 8 Evaluation

Everything below ran on one machine: 12 Neoverse-N1 cores, 46.9 GiB, Linux 6.17.0-1020-oracle aarch64, compiler 1.94.1 at the pinned toolchain. Percentiles interpolate between order statistics.

**Two kernels in separate processes.** Two kernels were brought up as separate operating-system processes, each generating its own keys and owning its own stores, issuers, agreements, leases, and receipt log, with a third process driving calls over iroh QUIC, relays disabled; the topology record reports 1 host. The co-signing and revocation hops are the shipped lanes unchanged; the request hop uses a transport defined for this experiment, since no shipped lane carries a tool call. Admitted calls: 100 calls, p50 206.803 ms, p99 234.520 ms at the driver, each costing 3 co-signing connections, counted by the receiver. Of that round trip the receiver’s evaluation window is 203.787 ms at the median and the driver’s evidence preparation 3.297 ms.

Denied calls: 7 scenarios driven over the wire, 140 calls, p50 58.821 ms, p99 88.462 ms. They return



Table 4: Local cost decomposition. The first two rows are complete kernel paths measured per invocation with both kernels in one process; the rest are components.

| Path or component                          | p50         | p99 or max   | samples |
|--------------------------------------------|-------------|--------------|---------|
| Admitted call, both kernels in one process | 13.863 ms   | 26.176 ms    | 200     |
| Refused before dispatch                    | 7.073 ms    | 18.646 ms    | 400     |
| Receipt append to the store                | 4681.643 us | 13566.644 us | 500     |
| Receipt signing                            | 229.340 us  | 270.205 us   | 100     |
| Receipt verification                       | 337.470 us  | 344.582 us   | 100     |
| Two-signature statement verification       | 339.127 us  | 350.904 us   | 100     |
| Cross-boundary admission decision          | 20.688 us   | 21.207 us    | 100     |

`chio_treaty_missing_scope`, `chio_treaty_scope_hash_mismatch`, `chio_treaty_missing_intersection`, `chio_treaty_intersection_mismatch`, `chio_treaty_missing_required_evidence`, `request_smuggled_trust_root`, and `request_smuggled_dynamic_trust`. In every one the receiver’s dispatch counter was zero: not one of these calls reached a tool. The latencies are reported because a denial slower than an allow is an availability problem.

Across the run 88 further calls were denied because the receiver’s snapshot had aged past its 500 ms freshness window while the origin publishes every 250 ms: 0 of the 100 attempts behind the admitted measurement, about 0 percent, 3 in the denial experiment, and 85 in the revocation experiment. They are the safety rule of Section 6 firing on the clock rather than on a revocation; they were retried and are excluded from the latency distributions.

Revoking at the origin and measuring to the receiver’s first denial took 322.899 ms at the median over 10 repeats, from 67.612 to 437.435 ms, a median of 2 calls. With the link cut the receiver serves until its snapshot leaves the freshness window and then denies, 744.681 ms at the median over 10 repeats, from 476.237 to 1093.769 ms, a median of 2 calls, counting a denial only once it persists for three further calls. Both medians sit under the  $k = 1$  bounds of Section 6, and every observation, both maxima included, sits under those bounds evaluated at the largest call count the run recorded and the admitted p99 latency, 4236.8 ms and 1453.6 ms. The partitioned receiver stops rather than falling back to its last good answer. On 3 of the 4 retained revoke records and 0 of the 10 cut records the denial’s pre-dispatch release could not be confirmed and was recorded as ambiguous rather than released, the path Section 4 names; the denial was unaffected and no call dispatched.

**What admission costs locally.** The shape of Table 4 is the finding: the cryptography is not the expense. Appending the receipt durably costs 13.8 times what verifying the two-signature statement costs, its schema and self-consistency rules included, and 226 times the receiver’s decision over already resolved evidence, where the comparisons of Table 3 run. Durability dominates, which is the price of Section 4’s ordering. The same admission across two processes costs 206.803 ms; the difference is the transport and the 3 co-signing connections, not the decision. One caveat belongs with the refusal row: its fixture refuses on a unanimous deny verdict, an early exit that never reaches the comparisons of Table 3 or the signature checks, so it is a lower bound on a refusal. Held at saturation for 60 s the same path admitted 3543 calls at 59.0 calls per second with 0 denials and 40.1 MiB of receipt-store growth.

The negative corpus is 20 in-process cases, each asserting its expected code; 8 run through the dispatch-capable hook with the tool-invocation counter asserted zero, and the remaining 12 exercise the verifier directly. Alongside it run 20 bilateral and 50 generic replay cases.

**Verification by someone who was not there.** The 50.6 KiB evidence package for the refund workflow verifies with no access to either kernel, every receipt, statement, lease, and governance record, in 49.7 ms at the median over 100 runs. A frozen corpus of 14 receipt cases holds four independent implementations, in Rust, TypeScript, Python, and Go, to the same canonical body bytes and verdict.

**Reproduction.** Every measurement above was taken at commit `041aa18374`. The cross-process run’s environment record marks its worktree clean; what differed were the retained result files written by the benchmark that ran before it, and no source file. Each run writes its environment record, raw samples, and the macro file this paper compiles from, so a printed and a retained number cannot drift.

## 9 Limitations and related work

The cross-organization run of Section 8 put the two kernels in separate processes on one host, not on two separately administered machines. That removes the shared address space, store, and key material, which are what the claim is about; it does not remove the shared operating system or clock source. A two-host run would, and the latencies would change. The rule does not depend on that experiment; the strength of the demonstration does. The corpus of Section 6 substitutes one field at a time; no case combines two.

Continuation consumption is at-most-once, so a crash between consume and dispatch loses the call rather than repeating it: right for a refund, an avoidable retry for an idempotent read. The chain closes in the kernel for one hop, since the next continuation in a longer chain is minted outside it. Denial paths below the verifier still return a rendered detail alongside the code, and separating the two everywhere is unfinished. Receipts are batched into Merkle checkpoints in the RFC 6962 shape [14] with offline inclusion proofs, and that is the whole claim: local audit evidence with verifiable inclusion, not a public append-only log, and with no anti-equivocation mechanism, so a kernel signing two contradictory checkpoints is detectable only by someone holding both.

**Related work.** Capability discipline without ambient authority is the tradition this protocol sits in [17, 23], saying exactly where a proof stops is seL4’s discipline [13], and attenuable credentials are old and good, from the trust-management line [9, 5] through macaroons [3] to the modern token families [22, 6], which carry caveats without a kernel, a decision record, or a price. Kerberos [20] crossed the boundary by transferring authority, workload identity by exchanging trust bundles and minting a local credential for a foreign workload [19], and token exchange by converting one issuer’s token into another’s [11]. All three move authority across the boundary, which is the step this protocol declines to take. Interchain light clients [10] solve pinned keys plus a monotone clock for chains rather than tool calls, and policy engines decide from a request they are handed and do not sign [7]. The envelope shapes we reuse [18, 21], transparent registration [4], and certificate transparency [14] answer how an artifact was built, who may register a statement, and how to publish verifiable records. What none of them defines is the object that is at once the authorization decision, the audit record, the accounting entry, and the evidence another organization’s kernel verifies against state it alone holds.

**What we believe.** The rule is smaller than the system that implements it, and we think it is the part that lasts. The receiver has to decide, from facts only it holds, and the record of that decision has to outlive the connection and be checkable by someone who trusts neither side.

## References

- [1] Agent2Agent Project. A2A: Agent2agent protocol specification. Specification, 2025.
- [2] Anthropic. Model context protocol specification. Specification, 2024.
- [3] Arnar Birgisson et al. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [4] Henk Birkholz et al. An architecture for trustworthy and transparent digital supply chains. IETF SCITT working group, Internet-Draft, 2025.
- [5] Matt Blaze et al. The KeyNote trust-management system version 2. RFC 2704, 1999.
- [6] Geoffroy Couprie and Clément Delafargue. Biscuit: A decentralized authorization token. Specification, v3, 2023.
- [7] Joseph W. Cutler et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2024.
- [8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction (CADE)*, 2021.
- [9] Carl M. Ellison et al. SPKI certificate theory. RFC 2693, 1999.
- [10] Christopher Goes. The interblockchain communication protocol: An overview. arXiv:2006.15918, 2020.
- [11] Michael B. Jones et al. OAuth 2.0 token exchange. RFC 8693, 2020.
- [12] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017.
- [13] Gerwin Klein et al. seL4: Formal verification of an OS kernel. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2009.
- [14] Ben Laurie et al. Certificate transparency. RFC 6962, IETF, 2013.
- [15] Anders Rundgren et al. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.
- [16] Jerome H. Saltzer et al. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
- [17] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [18] Secure Systems Lab. DSSE: Dead simple signing envelope. Secure Systems Lab specification, v1.0, 2021.
- [19] SPIFFE Project. SPIFFE: Secure production identity framework for everyone. Specification, federation and trust bundles, 2024.
- [20] Jennifer G. Steiner et al. Kerberos: An authentication service for open network systems. In *USENIX Winter Conference*, 1988.
- [21] Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.
- [22] UCAN Working Group. UCAN: User controlled authorization networks. Specification, v1.0, 2025.
- [23] Robert N. M. Watson et al. Capsicum: Practical capabilities for UNIX. In *USENIX Security Symposium*, 2010.